

PART 3: Advanced Collection Framework

3.1 ArrayList — Internals, Resizing, and Complexity

Definition & Internal Mechanics

`ArrayList` is a resizable array implementation of `List`. Internally, it is backed by an `Object[]` array. When you create an `ArrayList`, the default initial capacity is **10** (or 0 in Java 8+ if using the no-arg constructor, growing lazily on first add).

Growth formula: When the array is full, `ArrayList` creates a new array with size calculated as:

$$\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} >> 1)$$

This is equivalent to $\text{oldCapacity} \times 1.5$ (rounded down). The old array is then copied to the new array using `Arrays.copyOf()` which calls `System.arraycopy()` — a native method.

Complexity summary:

Operation	Average	Worst Case	Notes
<code>get(index)</code>	$O(1)$	$O(1)$	Direct array indexing
<code>add(element)</code>	$O(1)$ amortized	$O(n)$	Occasional resize
<code>add(index, element)</code>	$O(n)$	$O(n)$	Elements must shift right
<code>remove(index)</code>	$O(n)$	$O(n)$	Elements must shift left
<code>contains(element)</code>	$O(n)$	$O(n)$	Linear scan
<code>size()</code>	$O(1)$	$O(1)$	Cached field

Why amortized $O(1)$ for add: Each element is moved at most $O(\log n)$ times during its lifetime. The total work is $O(n \log n)$ for n inserts $\rightarrow O(\log n)$ amortized per insert. More precisely: summing the geometric series of copy costs gives $O(n)$ total for n inserts $\rightarrow O(1)$ amortized.

Real-Life Analogy

An `ArrayList` is like an office that rents cubicles. When they run out of space, they move to a bigger office 1.5× the size and bring all employees (copy the array). Accessing employee #42 (by

index) is instant since you know exactly where desk #42 is. But inserting a new employee in the middle means everyone after their desk has to shift one seat over.

Interview Context & Why It Matters

Understanding the growth strategy and its amortized analysis is crucial for capacity planning in high-throughput systems. Pre-sizing with `new ArrayList<>(expectedCapacity)` avoids repeated resizes and memory churning.

Production-Ready Code Snippet

```
java
```

```

import java.util.ArrayList;
import java.util.List;

/**
 * Demonstrates ArrayList internals: capacity growth, pre-sizing,
 * mid-list insertion cost, and iterator invalidation (fail-fast).
 */
public class ArrayListInternalsDemo {

    public static void main(String[] args) throws Exception {
        // Pre-sized ArrayList: avoids multiple resize operations
        List<String> optimized = new ArrayList<>(1_000_000); // Reserve capacity upf

        // Default: starts small, resizes: 10 → 15 → 22 → 33 → ... for 1M elements
        List<String> unoptimized = new ArrayList<>();

        long start = System.nanoTime();
        for (int i = 0; i < 100_000; i++) optimized.add("item-" + i);
        long optimizedTime = System.nanoTime() - start;

        start = System.nanoTime();
        for (int i = 0; i < 100_000; i++) unoptimized.add("item-" + i);
        long unoptimizedTime = System.nanoTime() - start;

        System.out.printf("Pre-sized:    %,d ns%n", optimizedTime);
        System.out.printf("Un-sized:    %,d ns%n", unoptimizedTime);

        // O(n) mid-list insertion demonstration
        List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));
        list.add(2, 99); // Insert at index 2: elements [3, 4, 5] all shift right
        System.out.println("After mid-insert: " + list); // [1, 2, 99, 3, 4, 5]

        // Fail-fast iterator: throws ConcurrentModificationException
        List<String> items = new ArrayList<>(List.of("a", "b", "c"));
        try {
            for (String item : items) {
                if (item.equals("b")) {
                    items.remove(item); // Modifies modCount during iteration
                }
            }
        } catch (java.util.ConcurrentModificationException e) {
            System.out.println("Caught ConcurrentModificationException – fail-fast t
        }
    }
}

```

```

// CORRECT way to remove during iteration: use Iterator.remove()
List<String> items2 = new ArrayList<>(List.of("a", "b", "c"));
java.util.Iterator<String> it = items2.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) {
        it.remove(); // Safe — updates modCount correctly
    }
}
System.out.println("After safe removal: " + items2); // [a, c]

// Or use removeIf (Java 8+) — cleanest approach
List<String> items3 = new ArrayList<>(List.of("a", "b", "c"));
items3.removeIf(s -> s.equals("b"));
System.out.println("After removeIf: " + items3); // [a, c]
}
}

```

Common Interview Question & Expert Answer

Q: "What's the time complexity of removing an element from the middle of an `ArrayList`, and how does that compare to `LinkedList`?"

A: "Removing from the middle of an `ArrayList` is $O(n)$ because all elements after the removed position must be shifted left by one position using `System.arraycopy()`. Despite the $O(n)$ algorithmic complexity, the operation is CPU-cache-friendly because it's a contiguous memory block copy — modern CPUs can execute this very fast via SIMD instructions. For `LinkedList`, the **removal itself** is $O(1)$ once you have the node reference (just update two pointers), but **finding** the node by index is $O(n)$ because you must traverse from the head or tail. So both operations end up $O(n)$ in the worst case. In practice, `ArrayList` is usually **faster** for most operations because of better CPU cache locality — linked list nodes are scattered in heap memory, causing cache misses on every next-pointer traversal. `LinkedList` only wins when you're doing many insertions at the **very beginning** or when you hold the iterator position."

3.2 LinkedList — Doubly-Linked Internals and CPU Cache Impact

Definition & Internal Mechanics

`LinkedList` implements both `List` and `Deque`. Internally, it's a **doubly-linked list** where each node (`Node<E>`) holds:

- `E item` — the data

- `Node<E> next` — pointer to next node
- `Node<E> prev` — pointer to previous node

The list maintains `first` and `last` pointers for $O(1)$ access to both ends.

Node overhead per element:

- 16 bytes: object header
- 8 bytes: `item` reference
- 8 bytes: `next` reference
- 8 bytes: `prev` reference
- = ~40 bytes per element vs ~4-8 bytes per reference in ArrayList

CPU Cache Problem:

- Modern CPUs have L1 cache (32KB), L2 cache (256KB), L3 cache (8-32MB)
- Cache line = 64 bytes — the CPU loads 64 bytes at once
- `ArrayList`: elements are adjacent in memory → loading one element likely loads neighboring elements into cache → **cache friendly**
- `LinkedList`: each node is a separate heap object at an arbitrary memory address → every `next` pointer dereference likely causes a **cache miss** → much slower in practice despite same $O(n)$ complexity

Real-Life Analogy

`ArrayList` is a bookshelf with books arranged by number — finding book 50 means going directly to slot 50, and flipping between adjacent books is fast because they're right next to each other. `LinkedList` is a treasure hunt — each book has a note inside saying "the next book is at [random house address across the city]." Technically you can get from book 1 to book 100, but each step requires a road trip.

Interview Context & Why It Matters

This question tests whether you understand **practical performance** beyond Big-O notation. Two $O(n)$ algorithms can differ by 10-100× in wall clock time due to cache behavior.

Production-Ready Code Snippet

```
java
```

```

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.LinkedList;
import java.util.List;

/**
 * Demonstrates LinkedList characteristics, Deque usage,
 * memory overhead, and cache miss impact via benchmarking.
 */
public class LinkedListDemo {

    public static void main(String[] args) {
        // LinkedList as Deque – O(1) at both ends
        Deque<String> deque = new LinkedList<>();
        deque.addFirst("middle");
        deque.addFirst("start");    // O(1) – updates 'first' pointer
        deque.addLast("end");        // O(1) – updates 'last' pointer
        System.out.println("Deque: " + deque); // [start, middle, end]
        System.out.println("Poll first: " + deque.pollFirst()); // start
        System.out.println("Poll last: " + deque.pollLast());    // end

        // Use ArrayDeque instead of LinkedList for queue/stack operations
        // ArrayDeque is faster in practice (no node allocation, cache friendly)
        Deque<String> betterDeque = new ArrayDeque<>();
        betterDeque.addFirst("a");
        betterDeque.addLast("b");

        // Benchmark: sequential traversal – ArrayList is faster due to cache locality
        int size = 500_000;
        List<Integer> arrayList = new ArrayList<>(size);
        List<Integer> linkedList = new LinkedList<>();

        for (int i = 0; i < size; i++) {
            arrayList.add(i);
            linkedList.add(i);
        }

        // Traversal benchmark
        long start = System.nanoTime();
        long sum1 = 0;
        for (int val : arrayList) sum1 += val;
        long arrayTime = System.nanoTime() - start;
    }
}

```

```

start = System.nanoTime();
long sum2 = 0;
for (int val : linkedList) sum2 += val; // Each next() dereferences a pointer
long linkedTime = System.nanoTime() - start;

System.out.printf("ArrayList traversal:    %,d ns (sum=%d)%n", arrayTime, sum2);
System.out.printf("LinkedList traversal:    %,d ns (sum=%d)%n", linkedTime, sum2);
// LinkedList typically 3-10x slower for traversal despite same O(n) complexity

// When LinkedList wins: frequent insertions at known positions via ListIterator
List<String> ll = new LinkedList<>();
for (int i = 0; i < 10; i++) ll.add("item-" + i);
java.util.ListIterator<String> lit = ll.listIterator(5); // position at middle
lit.add("INSERTED"); // O(1) after positioning – no array shift
System.out.println("After mid-insert via iterator: size=" + ll.size());
}
}

```

Common Interview Question & Expert Answer

Q: "When would you actually choose `LinkedList` over `ArrayList`? And is `LinkedList` ever the best choice for a stack or queue?"

A: "Honestly, for most real-world use cases, `ArrayList` or `ArrayDeque` is the better choice. `LinkedList` only makes sense when: (1) You have a `ListIterator` positioned in the middle and need to insert/delete many elements there repeatedly — the actual pointer manipulation is $O(1)$ once positioned. (2) You need a true doubly-linked list with stable node references for algorithms that depend on node addresses (like a scheduler or LRU cache). For **stack** or **queue**, `ArrayDeque` is strictly better than `LinkedList` — it's backed by a circular array, so `addFirst()`/`addLast()`/`pollFirst()`/`pollLast()` are all amortized $O(1)$, it has no per-element node allocation overhead (~40 bytes/node for `LinkedList` vs ~4-8 bytes per slot in `ArrayDeque`), and its contiguous memory layout means far fewer cache misses. The Java documentation itself recommends `ArrayDeque` over `LinkedList` for queue operations. In practice, I would only choose `LinkedList` if I specifically needed the `Deque` interface AND had profiling evidence that `ArrayDeque` was insufficient."

3.3 Set Variants — `HashSet`, `LinkedHashSet`, `TreeSet`

Definition & Internal Mechanics

HashSet:

- Backed by a `HashMap<E, Object>` where the dummy value is `static final Object PRESENT = new Object()`
- `add(e)` $\rightarrow$ `map.put(e, PRESENT)` — returns `false` if key already existed
- All `HashMap` mechanics apply: $O(1)$ average for `add`, `remove`, `contains`
- No ordering guarantee

LinkedHashSet:

- Backed by a `LinkedHashMap` — a `HashMap` augmented with a doubly-linked list running through all entries in **insertion order**
- Maintains insertion order for iteration at the cost of extra memory (two extra pointers per entry)
- $O(1)$ for `add`, `remove`, `contains`

TreeSet:

- Backed by a `TreeMap` — implemented as a **Red-Black Tree** (self-balancing BST)
- Maintains elements in **natural order** (`Comparable`) or a custom `Comparator`
- $O(\log n)$ for `add`, `remove`, `contains`
- Provides `floor()`, `ceiling()`, `headSet()`, `tailSet()`, `subSet()` operations

Red-Black Tree properties:

- Every node is red or black
- Root is black
- Red nodes cannot have red children (no two adjacent red nodes)
- Every path from a node to a leaf has the same number of black nodes
- These constraints guarantee tree height $\leq 2 \log_2(n + 1) \rightarrow O(\log n)$ guaranteed

Real-Life Analogy

- **HashSet** = a lottery ball machine: balls are tossed in at random positions, but you can instantly check if a ball is there.
- **LinkedHashSet** = a VIP guest list in arrival order: you can check instantly if someone's on it, AND see them in the order they arrived.
- **TreeSet** = a sorted library catalog: takes more effort to add a book (it must go in alphabetical order, requiring catalog reshuffling), but you can do sorted range queries like "all books from A to M."

Interview Context & Why It Matters

Interviewers ask candidates to choose the right Set implementation based on ordering requirements and performance constraints. A wrong choice (e.g., `TreeSet` when ordering isn't needed) costs $O(\log n)$ per operation vs $O(1)$.

Production-Ready Code Snippet

```
java
```

```

import java.util.*;

/**
 * Demonstrates HashSet, LinkedHashSet, and TreeSet internals,
 * Red-Black Tree ordering, and practical usage scenarios.
 */
public class SetVariantsDemo {

    record Product(String id, String name, double price) implements Comparable<Product> {
        @Override
        public int compareTo(Product other) {
            return Double.compare(this.price, other.price); // Natural order: by price
        }
    }

    public static void main(String[] args) {
        // HashSet – O(1) operations, no order guarantee
        Set<String> hashSet = new HashSet<>();
        hashSet.add("banana");
        hashSet.add("apple");
        hashSet.add("cherry");
        hashSet.add("apple"); // duplicate – ignored, returns false
        System.out.println("HashSet (unordered): " + hashSet);
        System.out.println("Contains 'apple': " + hashSet.contains("apple")); // O(1)

        // LinkedHashSet – preserves insertion order
        Set<String> linkedSet = new LinkedHashSet<>();
        linkedSet.add("banana");
        linkedSet.add("apple");
        linkedSet.add("cherry");
        linkedSet.add("apple"); // ignored
        System.out.println("LinkedHashSet (insertion order): " + linkedSet);
        // Output: [banana, apple, cherry] – always in insertion order

        // TreeSet – sorted by natural order (or Comparator)
        Set<String> treeSet = new TreeSet<>();
        treeSet.add("banana");
        treeSet.add("apple");
        treeSet.add("cherry");
        System.out.println("TreeSet (sorted): " + treeSet); // [apple, banana, cherry]

        // TreeSet range operations – unique to sorted sets
        NavigableSet<Integer> numbers = new TreeSet<>(Set.of(1, 3, 5, 7, 9, 11, 13))
    }
}

```

```

System.out.println("HeadSet (< 7): " + numbers.headSet(7));           // [1, 3
System.out.println("TailSet (>= 7): " + numbers.tailSet(7));           // [7, 9
System.out.println("SubSet [5, 11): " + numbers.subSet(5, 11));         // [5, 7
System.out.println("Floor of 6: " + numbers.floor(6));                  // 5 (la
System.out.println("Ceiling of 6: " + numbers.ceiling(6));              // 7 (sm

// TreeSet with custom Product Comparator (by price, then by id for stabilit
Comparator<Product> byPrice = Comparator.comparingDouble(Product::price)
                                           .thenComparing(Product::id);
TreeSet<Product> productSet = new TreeSet<>(byPrice);
productSet.add(new Product("P003", "Keyboard", 79.99));
productSet.add(new Product("P001", "Mouse", 29.99));
productSet.add(new Product("P002", "Monitor", 349.99));
productSet.add(new Product("P004", "Webcam", 79.99)); // same price as Keybo

System.out.println("Products by price:");
productSet.forEach(p -> System.out.printf(" %s: $%.2f%n", p.name(), p.price

// Set operations using built-in collection methods
Set<Integer> setA = new HashSet<>(Set.of(1, 2, 3, 4, 5));
Set<Integer> setB = new HashSet<>(Set.of(3, 4, 5, 6, 7));

Set<Integer> intersection = new HashSet<>(setA);
intersection.retainAll(setB); // Modifies in-place – only keeps common eleme
System.out.println("Intersection: " + intersection); // [3, 4, 5]

Set<Integer> union = new HashSet<>(setA);
union.addAll(setB);
System.out.println("Union: " + union); // [1, 2, 3, 4, 5, 6, 7]

Set<Integer> difference = new HashSet<>(setA);
difference.removeAll(setB);
System.out.println("Difference (A - B): " + difference); // [1, 2]
}
}

```

Common Interview Question & Expert Answer

Q: "How does `TreeSet` maintain sorted order internally? And what happens to the tree if many elements are inserted in already-sorted order?"

A: "`TreeSet` is backed by a `TreeMap` which uses a **Red-Black Tree** — a self-balancing Binary Search Tree. When you insert an element, it finds the correct BST position in $O(\log n)$ time, then

performs **rotations** and **recoloring** to maintain the Red-Black invariants (no two adjacent red nodes, equal black-height for all paths to leaves). These rotations ensure the tree stays balanced with height $O(\log n)$ even in adversarial insertion orders. If many elements arrive in sorted order in a plain BST, it would degenerate to a linked list (height $O(n)$, $O(n)$ lookup). The Red-Black Tree prevents this — sorted insertion just triggers more right rotations but the tree stays balanced. The trade-off vs a plain BST is the overhead of rotation operations and color bits, but the guarantee of $O(\log n)$ worst case makes it worth it for production use. This is why `TreeSet.contains()` is guaranteed $O(\log n)$ while a poorly-implemented BST could be $O(n)$."

3.4 HashMap Internal Mechanics — The Deep Dive

Definition & Internal Mechanics

`HashMap` is the most important data structure to understand deeply in Java interviews.

Internal Structure: An array of `Node<K, V>[]` (called `table`). Each array slot (bucket) holds either `null`, a single `Node`, or a linked list of nodes (for collisions), which may be converted to a **Red-Black Tree** when a bucket becomes too long.

Hashing pipeline:

Step 1: Get key's `hashCode()` Step 2: Apply hash spreading:

$$\text{hash}(\text{key}) = \text{hashCode} \oplus (\text{hashCode} \ggg 16)$$

(XOR upper 16 bits with lower 16 bits — spreads entropy to prevent clustering)

Step 3: Compute bucket index:

$$\text{index} = (\text{capacity} - 1) \& \text{hash}$$

(Bitwise AND instead of modulo — valid because capacity is always a power of 2)

Collision Resolution:

- Same bucket → traverse the linked list comparing keys via `equals()`
- When a bucket's chain length exceeds **TREEIFY_THRESHOLD = 8**, and table capacity $\geq$ **MIN_TREEIFY_CAPACITY = 64**, the chain is converted to a Red-Black Tree → lookup degrades from $O(n)$ to $O(\log n)$
- When a bucket's tree size drops below **UNTREEIFY_THRESHOLD = 6** (during removal), it reverts to a linked list

Load Factor and Rehashing:

- Default load factor = **0.75**
- Resize trigger: when $\text{size} > \text{capacity} \times \text{loadFactor}$
- On resize: capacity **doubles** (always powers of 2), all entries are **rehashed** — new bucket index = old index OR old index + old capacity (due to the power-of-2 property)
- Rehashing is $O(n)$ — expensive

Complexity:

Operation	Average	Worst Case (all keys collide)
<code>get(key)</code>	$O(1)$	$O(\log n)$ (with treeification)
<code>put(key, value)</code>	$O(1)$	$O(\log n)$
<code>remove(key)</code>	$O(1)$	$O(\log n)$

Null key: `HashMap` allows one `null` key — always stored in bucket 0 (since $\text{hash}(\text{null}) = 0$).

Real-Life Analogy

A `HashMap` is like a **post office with numbered mailboxes**. When you send mail (put a key-value pair), the post office runs your recipient name through a formula (hash function) to determine which mailbox slot (bucket) to use. If two people get assigned the same slot (collision), the post office chains their letters together on a hook. If one slot gets too many letters (>8), they install a faster-access filing system (Red-Black Tree). When the building runs out of mailboxes (load factor exceeded), they move to a $2\times$ bigger building and re-sort everything.

Interview Context & Why It Matters

`HashMap` internals are asked in nearly 70% of senior Java interviews. Understanding every layer — hashing, spreading, treeification, rehashing — separates senior engineers from intermediates.

Production-Ready Code Snippet

```
java
```

```

import java.util.*;

/**
 * Deep dive into HashMap: custom hashCode impact,
 * treeification simulation, iteration order, and performance comparison
 * between HashMap, LinkedHashMap, and TreeMap.
 */
public class HashMapInternalsDemo {

    // Custom key class – demonstrates good vs. bad hashCode
    static class GoodKey {
        private final String value;

        public GoodKey(String value) {
            this.value = Objects.requireNonNull(value);
        }

        @Override
        public boolean equals(Object obj) {
            if (!(obj instanceof GoodKey other)) return false;
            return Objects.equals(value, other.value);
        }

        @Override
        public int hashCode() {
            return Objects.hash(value); // Well-distributed hash
        }

        @Override
        public String toString() { return "Key(" + value + ")"; }
    }

    // BAD hashCode: always returns the same value – forces all keys into one bucket
    // This degrades HashMap to a linked list: O(n) per operation!
    static class BadKey {
        private final String value;

        public BadKey(String value) { this.value = value; }

        @Override
        public boolean equals(Object obj) {
            if (!(obj instanceof BadKey other)) return false;
            return Objects.equals(value, other.value);
        }
    }
}

```

```

    }

    @Override
    public int hashCode() { return 42; } // CATASTROPHIC – all keys collide!
}

public static void main(String[] args) {
    // HashMap iteration order is NOT guaranteed
    Map<String, Integer> hashMap = new HashMap<>();
    hashMap.put("banana", 2);
    hashMap.put("apple", 5);
    hashMap.put("cherry", 1);
    System.out.println("HashMap (no order): " + hashMap);

    // LinkedHashMap: maintains insertion order
    Map<String, Integer> linkedMap = new LinkedHashMap<>();
    linkedMap.put("banana", 2);
    linkedMap.put("apple", 5);
    linkedMap.put("cherry", 1);
    System.out.println("LinkedHashMap (insertion order): " + linkedMap);

    // LinkedHashMap with access order – useful for LRU caches
    Map<String, Integer> accessOrderMap = new LinkedHashMap<>(16, 0.75f, true) {
        // Override removeEldestEntry for bounded LRU cache behavior
        @Override
        protected boolean removeEldestEntry(Map.Entry<String, Integer> eldest) {
            return size() > 3; // Keep max 3 entries
        }
    };
    accessOrderMap.put("a", 1);
    accessOrderMap.put("b", 2);
    accessOrderMap.put("c", 3);
    accessOrderMap.get("a"); // 'a' now most recently accessed
    accessOrderMap.put("d", 4); // 'b' is eldest – gets evicted
    System.out.println("LRU cache (max 3): " + accessOrderMap); // {c=3, a=1, d=4}

    // TreeMap: sorted by key
    Map<String, Integer> treeMap = new TreeMap<>();
    treeMap.put("banana", 2);
    treeMap.put("apple", 5);
    treeMap.put("cherry", 1);
    System.out.println("TreeMap (sorted): " + treeMap); // {apple=5, banana=2, cherry=1}

    // Performance impact of bad hashCode

```

```

int count = 100_000;
Map<GoodKey, Integer> goodMap = new HashMap<>(count * 2);
Map<BadKey, Integer> badMap = new HashMap<>();

long start = System.nanoTime();
for (int i = 0; i < count; i++) goodMap.put(new GoodKey("key-" + i), i);
long goodPut = System.nanoTime() - start;

start = System.nanoTime();
// Only use 1000 for bad map - 100k would take minutes
for (int i = 0; i < 1_000; i++) badMap.put(new BadKey("key-" + i), i);
long badPut = System.nanoTime() - start;

System.out.printf("GoodKey HashMap put 100k: %,d ns%n", goodPut);
System.out.printf("BadKey HashMap put 1k: %,d ns (extrapolate for comparison)", badPut);

// HashMap compute/merge operations - atomic-style updates
Map<String, Integer> wordCount = new HashMap<>();
String[] words = {"the", "cat", "sat", "on", "the", "mat", "the"};
for (String word : words) {
    wordCount.merge(word, 1, Integer::sum); // Elegant word count
}
System.out.println("Word counts: " + wordCount); // {the=3, cat=1, sat=1, on=1, mat=1}

wordCount.compute("the", (k, v) -> v == null ? 1 : v + 10);
System.out.println("After compute: " + wordCount.get("the")); // 13

wordCount.computeIfAbsent("new_word", k -> k.length()); // Only if key absent
System.out.println("ComputeIfAbsent: " + wordCount.get("new_word")); // 8
}
}

```

Common Interview Question & Expert Answer

Q: "Why does `HashMap`'s resize always double capacity, and why does capacity have to be a power of 2? What happens to entries during rehash?"

A: "Capacity must always be a power of 2 because of how the bucket index is computed:

`(capacity - 1) & hash`. When capacity is a power of 2, `(capacity - 1)` is a bitmask of all 1s (e.g., capacity 16 → mask `0b1111`). This bitwise AND is equivalent to `hash % capacity` but executes in a single CPU clock cycle vs. a division instruction which is much slower. If capacity were not a power of 2, the AND operation would not be equivalent to modulo. Doubling is chosen because it preserves a beautiful property during rehash: an entry's new bucket index is **either its**

old index or its old index + old capacity. This is because adding one bit to the mask either leaves the bit as 0 (same bucket) or as 1 (bucket + old capacity). This means roughly half the entries stay in their current bucket and half move to `oldIndex + oldCapacity`. No entry needs a full rehash computation — the JVM can determine the new bucket from a single bit check. This is why Java's HashMap rehash is faster than implementations that require full modulo recomputation."

3.5 Fail-Fast vs. Fail-Safe Iterators

Definition & Internal Mechanics

Fail-Fast Iterators:

- Used by `ArrayList`, `HashMap`, `HashSet`, `LinkedList`, etc.
- Each collection maintains a `modCount` field (modification count) — incremented on every structural change (`add`, `remove`, `clear`, etc.)
- When an iterator is created, it captures the current `modCount` as `expectedModCount`
- On every `next()` call, the iterator checks: if `modCount != expectedModCount`, throw `ConcurrentModificationException`
- **Best-effort:** The spec says this is not guaranteed and should not be relied upon for correctness — it's a debugging aid, not a security mechanism

Fail-Safe Iterators:

- Used by `ConcurrentHashMap`, `CopyOnWriteArrayList`, `CopyOnWriteArraySet`
- Operate on a **snapshot** or **cloned** copy of the data
- Changes made to the collection after iterator creation are NOT visible to the iterator
- Never throw `ConcurrentModificationException`
- Trade-off: may miss recent updates; `CopyOnWriteArrayList` creates a full copy on every write — $O(n)$ write cost

Feature	Fail-Fast	Fail-Safe
Exception	<code>ConcurrentModificationException</code>	None
Snapshot	No — live data	Yes — snapshot
Thread-safe	No	Yes
Memory overhead	Low	High (copy)
Updated during iteration	Yes (throws)	No (stale copy)
Examples	<code>ArrayList</code> , <code>HashMap</code>	<code>ConcurrentHashMap</code> , <code>CopyOnWriteArrayList</code>

Real-Life Analogy

Fail-Fast is like a document review meeting — if someone edits the document on the projector while you're reading a paragraph, you shout "Stop! You changed it while I was reading!" and the review halts. **Fail-Safe** is like reviewing a photocopy of the document — someone can edit the original during your review, and you won't notice or care since you're working off your copy.

Interview Context & Why It Matters

This is a critical concurrency question. In multi-threaded code, using a fail-fast iterator while another thread modifies the collection is a serious bug. Interviewers want to see you know the safe alternatives and their trade-offs.

Production-Ready Code Snippet

java

```

import java.util.*;
import java.util.concurrent.*;

/**
 * Demonstrates fail-fast vs fail-safe iterators,
 * safe modification patterns, and concurrent collection behavior.
 */
public class IteratorDemo {

    public static void main(String[] args) throws InterruptedException {
        // Fail-fast: ConcurrentModificationException
        List<String> list = new ArrayList<>(List.of("a", "b", "c", "d"));
        try {
            for (String s : list) {
                System.out.print(s + " ");
                if (s.equals("b")) list.remove(s); // Modifies modCount during for-each
            }
        } catch (ConcurrentModificationException e) {
            System.out.println("\nFail-fast triggered: " + e.getClass().getSimpleName());
        }

        // Safe pattern 1: collect then remove
        List<String> list2 = new ArrayList<>(List.of("a", "b", "c", "d"));
        List<String> toRemove = new ArrayList<>();
        for (String s : list2) {
            if (s.equals("b") || s.equals("d")) toRemove.add(s);
        }
        list2.removeAll(toRemove);
        System.out.println("After safe remove: " + list2);

        // Safe pattern 2: removeIf (Java 8+) – cleanest
        List<String> list3 = new ArrayList<>(List.of("a", "b", "c", "d"));
        list3.removeIf(s -> s.equals("b") || s.equals("d"));
        System.out.println("After removeIf: " + list3);

        // Fail-safe: CopyOnWriteArrayList – never throws CME
        List<String> cowList = new CopyOnWriteArrayList<>(List.of("a", "b", "c", "d"));
        for (String s : cowList) {
            System.out.print(s + " ");
            if (s.equals("b")) cowList.add("X"); // Modifies a NEW copy – iterator not affected
        }
        System.out.println("\nCOW iteration complete. List: " + cowList);
        // "X" was added but not seen during the iteration above
    }
}

```

```

// ConcurrentHashMap: fail-safe-like – weakly consistent iterator
Map<String, Integer> concurrentMap = new ConcurrentHashMap<>();
concurrentMap.put("a", 1);
concurrentMap.put("b", 2);
concurrentMap.put("c", 3);

// Iterator is weakly consistent: may or may not see updates made during iteration
// but will NEVER throw ConcurrentModificationException
for (Map.Entry<String, Integer> entry : concurrentMap.entrySet()) {
    System.out.print(entry.getKey() + "=" + entry.getValue() + " ");
    concurrentMap.put("d", 4); // Safe – no CME
}
System.out.println("\nConcurrentHashMap final size: " + concurrentMap.size());
}
}

```

Common Interview Question & Expert Answer

Q: "In a single-threaded context, can a fail-fast iterator throw `ConcurrentModificationException`? How would you safely remove elements that match a condition from an `ArrayList` during iteration?"

A: "Yes — fail-fast applies to ANY structural modification to the collection while iterating, not just modifications from other threads. In a single thread, if you call `list.remove(element)` or `list.add(element)` inside a for-each loop over the same list, the `modCount` is incremented but the iterator's `expectedModCount` is not updated, causing `ConcurrentModificationException`. The safest approach in modern Java is `list.removeIf(predicate)` — it's an atomic in-place modification that uses an optimized two-pointer algorithm internally. Alternative: use `Iterator.remove()` which does update `expectedModCount`. For replacing elements (not just removing), use `ListIterator.set()`. For complex transformations, collect indices to modify, then apply changes after the loop finishes. `CopyOnWriteArrayList` avoids the issue entirely but costs $O(n)$ per write, so it's only appropriate for read-heavy workloads with rare modifications — like maintaining a list of event listeners."